TECHNICAL DESIGN DOCUMENT

# Invoice-Guard

## Design Documentation

Supplier invoice PDF to exactly one ERP posting. A model only reads the invoice; a deterministic rule engine decides whether it can post; anything uncertain goes to a person.

| | |
|---|---|
| **Version** | 2.0 |
| **Date** | 8 November 2026 |
| **Author** | Buddhika Amarasinghe |
| **Based on** | Invoice Validation Pipeline TSD v2 (4 November 2026), Phase 1 |
| **Repository** | github.com/BudAma87/invoice-guard |
| **Code baseline** | `main` at `ae38d23` (includes the security fixes, PR #8) |

# Document history

| Version | Date | Changes |
|---|---|---|
| 1.0 | 7 Nov 2026 | Phase 1: pipeline, rule engine, review API and UI, SLA escalation, 79 tests |
| 2.0 | 8 Nov 2026 | Daily reconciliation; reconciliation tables and migrations; full API and configuration reference; security controls and both security assessments; 117 tests; updated architecture diagram, gaps and next steps |

# Contents

1. Overview and scope

2. Architecture

3. Data model and state machine

4. Rule engine

5. Exactly-once posting and failure handling

6. Review UI/UX

7. Human controls and security

8. Implementation and testing

9. Problems found while building, and their fixes

10. Decisions, known gaps and next steps

# 1. Overview and scope

Invoice-Guard turns each supplier invoice PDF into exactly one ERP posting. A model only reads the invoice; a deterministic rule engine decides whether it can post; anything uncertain goes to a person. It implements Phase 1 of the Invoice Validation Pipeline TSD v2 (Nov 4, 2026). Phase 1, SLA escalation, daily reconciliation and the security hardening from two assessments are all merged to `main` (PRs #1, #6, #7 and #8).

## Goals the design serves

- **No double posting.** Exactly one ERP document per invoice, even though Kafka delivers at least once.
- **No lost invoice.** Every message ends Posted, Rejected or Duplicate, with an audit trail.
- **Uncertain means human.** Any failed rule or field confidence below 0.85 goes to review, never straight to the ERP.
- **Fast, trustworthy review.** A reviewer clears a typical exception in under 2.5 minutes without leaving the screen.

## Phase 1 status

| Area | State |
|---|---|
| Rule engine, state DB, inbox, outbox relay, review API, review UI, SLA escalation scheduler, daily reconciliation | Built and tested (117 automated tests) |
| Security hardening: fail-closed auth, signed links, headers, rate limit, CI security scans | Built and tested; merged in PR #8 (section 7) |
| Fake ERP with fault injection, demo seed, walkthrough | Built |
| SQLite (dev and pilot) | Tested |
| Kafka consumer, Kafka message ledger, Anthropic extractor, Postgres | Wired, not yet exercised against real systems |
| SSO, email/Teams delivery of SLA notifications, pager integration, extraction eval scoring | Not built (section 10) |

Out of scope, as in the TSD: payment runs, supplier onboarding, PO creation, OCR model training and ERP-side approval workflows.

# 2. Architecture

The model only reads the invoice; deterministic code decides whether it posts, and the only way into the ERP is the outbox relay, reached by a clean auto-pass or a person's approval. Every step writes to the state DB before the next starts, so a crash resumes from the last committed step.

**Only the outbox relay talks to the ERP; scheduled checks watch the rest**

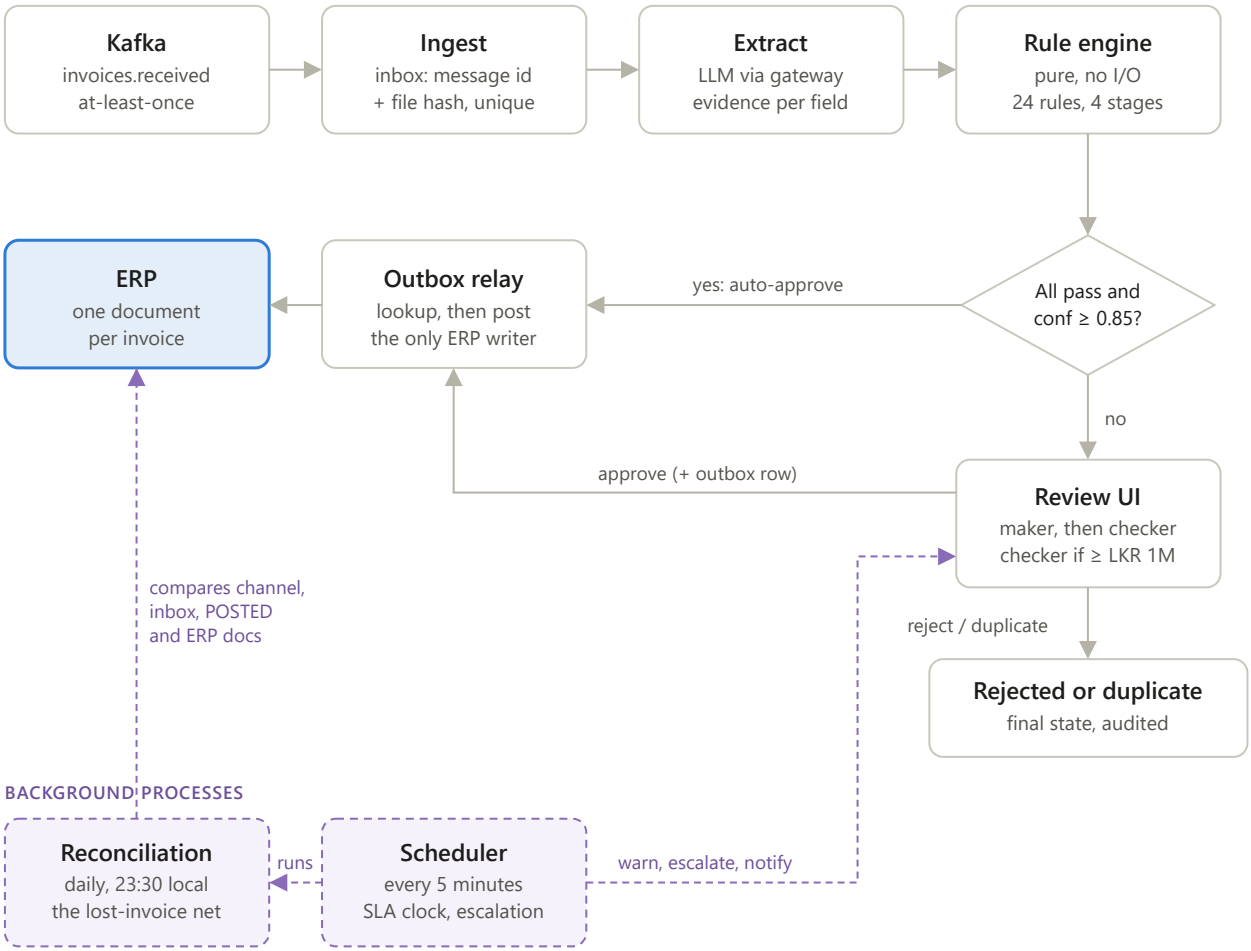



*Figure 1. Message-to-ERP flow: 7 pipeline components, 1 decision, and the 2 background processes that watch them.*

A clean invoice goes straight from the rule engine to the relay; anything else waits for a person, and only an approval writes the outbox row the relay drains.

## Components

| Component | Process | Responsibility |
|---|---|---|
| Ingest | `services/ingest` | Kafka consumer; writes the inbox row, then commits the offset |
| Worker | `services/worker` | Leases RECEIVED invoices from the DB; extract, normalise, enrich, validate, route; sweeper for expired leases |
| LLM gateway | `libs/llm_gateway` | Structured outputs, shared token bucket, circuit breaker, repair re-prompt, fallback model |

| Component | Process | Responsibility |
|---|---|---|
| Rule engine | `libs/rules` | Pure, versioned validation; shared by worker and API |
| Review API | `services/api` | Queue, invoice, validate, decision, check, ops and reconciliation endpoints; security headers and rate limit |
| Review UI | `apps/web` | Queue, workspace, checker view, ops dashboard with daily reconciliation panel |
| Outbox relay | `services/relay` | Drains the outbox; ERP lookup before every post |
| ERP adapter | `libs/erp_adapter` | Protocol with lookup by external reference; fake ERP for dev |
| Object store | `libs/core/store.py` | PDFs by SHA-256, write-once; signed 5-minute URLs with a dedicated key; hash-only keys |
| Scheduler | `services/scheduler` | Every 5 minutes: SLA clock in business hours (paused on hold), escalation once per level, priority recompute, notification delivery; daily reconciliation after 23:30 local |
| Reconciliation | `libs/core/reconcile.py` | Daily lost-invoice net: channel vs inbox, POSTED vs ERP, ERP vs POSTED; stores, audits and pages differences |
| Message ledger | `libs/core/ledger.py` | What the channel delivered, read independently of the inbox: Kafka topic (prod) or ledger file (dev) |

## Repository layout

| Path | Holds |
|---|---|
| `apps/web/` | Next.js review UI: queue, workspace, checker view, ops dashboard |
| `services/ingest/` | Kafka consumer to inbox (dedupe by message id and file hash) |
| `services/worker/` | Extract, normalise, enrich, validate, route; lease sweeper |
| `services/api/` | Review, validate, decision, check, ops and reconciliation endpoints |
| `services/relay/` | Outbox to ERP adapter (the only ERP writer) |
| `services/scheduler/` | Every 5 minutes: SLA escalation; daily reconciliation after 23:30 |
| `libs/rules/` | Pure rule engine and versioned rule sets |
| `libs/core/` | Normalisation, enrichment, routing, SLA, escalation, review workflow, reconciliation, message ledger, object store |
| `libs/erp_adapter/` | ERP protocol and fake ERP with fault injection |
| `libs/llm_gateway/` | Extraction schema and prompt, rate limiter, circuit breaker |
| `libs/db/` | Models, state machine, audit chain, Alembic migrations 0001–0003 |
| `scripts/` | Demo seed, PDF generator, walkthrough, GitHub setup |
| `tests/` | Rules, pipeline, reconciliation, SLA, review API, security |
| `evals/` | Labelled invoices and scoring (Phase 2) |
| `docs/` | TSD v1 and v2, this design document, screenshots |
| `.github/` | CI (backend, web, security), Dependabot, CODEOWNERS, templates |

# 3. Data model and state machine

The state DB is the source of truth for every invoice; Kafka is only the delivery channel and the ERP holds only the final result. Every state change is one transaction, guarded by a `version` column (optimistic locking), and writes one append-only, hash-chained event.

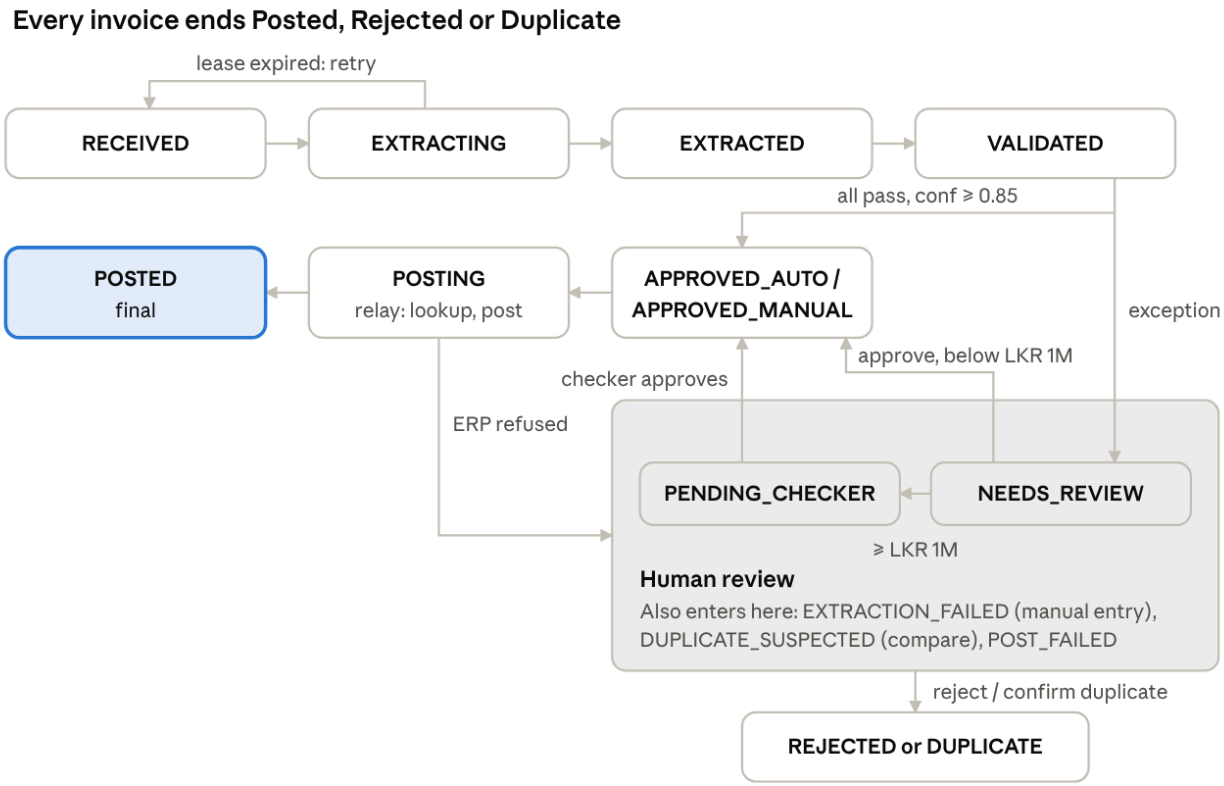



*Figure 2. Invoice state machine, main paths.*

The pipeline runs left to right along the top; anything it cannot decide drops into human review, and only an approval leads back up to posting. The table below lists every allowed transition (`libs/db/state.py`).

## Invoice states

| State | Meaning | Next |
|---|---|---|
| RECEIVED | Inbox written, PDF stored | EXTRACTING |
| EXTRACTING | Model call in progress (120 s lease) | EXTRACTED, EXTRACTION_FAILED, RECEIVED (retry) |
| EXTRACTED | Parsed and normalised | VALIDATED |
| EXTRACTION_FAILED | Retries exhausted; manual-entry task | NEEDS_REVIEW, REJECTED |
| VALIDATED | Rules evaluated | APPROVED_AUTO, NEEDS_REVIEW, DUPLICATE_SUSPECTED |
| DUPLICATE_SUSPECTED | Business key already live | DUPLICATE, NEEDS_REVIEW, REJECTED |

| State | Meaning | Next |
|---|---|---|
| NEEDS_REVIEW | In a person's queue | APPROVED_MANUAL, PENDING_CHECKER, REJECTED, DUPLICATE |
| PENDING_CHECKER | Maker done, at or above LKR 1M | APPROVED_MANUAL, NEEDS_REVIEW (sent back) |
| APPROVED_AUTO / APPROVED_MANUAL | Outbox row written | POSTING |
| POSTING | Relay calling the ERP | POSTED, POST_FAILED, back to APPROVED_* to wait |
| POST_FAILED | ERP refused (business error) | NEEDS_REVIEW |
| POSTED · REJECTED · DUPLICATE | Final | — |

## Core tables

| Table | Holds | Unique constraints |
|---|---|---|
| `inbox` | message id, PDF hash, source, invoice id | `message_id`; `pdf_sha256` |
| `invoice` | Header in minor units, state, version, confidence and evidence per field, business key, amount in LKR, rule-set version, posting period | `business_key` where state ≠ REJECTED (partial index) |
| `invoice_line` | Description, item code, UoM, PO line, qty (exact decimal text), unit price, amount, confidence, evidence | (`invoice_id`, `line_no`) |
| `extraction` | Attempt, model, prompt version, parsed output, latency, tokens, error | (`invoice_id`, `attempt`) |
| `rule_result` | One row per rule per evaluation, with expected, actual, message, fields | (`invoice_id`, `rule_key`, `evaluation_id`) |
| `review_task` | Kind, reasons, priority, assignee and lease, SLA due, hold (start, end, paused business seconds), escalation level and pool, maker, checker, corrections, overrides, results before and after | One open task per invoice (partial index) |
| `invoice_event` | Append-only audit: action, from/to state, actor, role, IP, payload, hash, previous hash | — |
| `outbox` | Idempotency key = invoice id, status (PENDING, UNKNOWN, DONE, FAILED), attempts, next attempt, ERP doc no. | `idempotency_key` |
| `notification` | SLA warning, breach and escalation messages: kind, recipients (user or role@entity), subject, body, status (PENDING, SENT, FAILED), attempts | (`task_id`, `kind`) |
| `reconciliation_run` | One row per entity, business day and attempt: trigger (schedule or manual), status (RUNNING, OK, DIFFERENCES, PARTIAL, ERROR), counts, skipped checks and why, difference count, error | — |
| `reconciliation_item` | One difference: kind, severity (critical or high), invoice id, message id, ERP doc no., detail | — |

The same SQLAlchemy models run on SQLite (WAL, single writer) and Postgres 16; Alembic owns migrations: `0001` initial schema, `0002` SLA escalation and notifications, `0003` daily reconciliation runs and items. Timestamps go through a `UTCDateTime` type so they are always timezone-aware UTC on both databases.

# 4. Rule engine

The rule engine is one pure function, `evaluate(invoice, context, rule_set) → [RuleResult]`, with no I/O. The pipeline and the review API call the same function, so a reviewer always sees exactly what the agent saw. Being pure also makes it replayable for audits and unit-testable at the TSD's boundaries.

**Evaluation order.** Rules run in four stages. Every rule in every stage runs, so the reviewer sees all problems at once. A rule missing an input returns `SKIPPED` and names its dependency (for example `needs LINE_PO_MATCH`).

| Stage | Rules | Card in the UI |
|---|---|---|
| 1. Shape | DATE_PARSE, LINES_PRESENT | Date, Lines |
| 2. Identity | SUPPLIER_RESOLVE, SUPPLIER_INACTIVE (BLOCK), DUP_EXACT (BLOCK), DUP_FUZZY, PO_OWNERSHIP | Supplier and duplicates |
| 3. Arithmetic | LINE_ARITH, SUBTOTAL_SUM, TAX_CHECK, TOTAL_RECONCILE, TOTAL_POSITIVE | Total, Lines |
| 4. Reference | DATE_NOT_FUTURE, DATE_AGE, DATE_AFTER_PO, DATE_PERIOD_OPEN, CURRENCY_MATCH, TOTAL_VS_PO, LINE_PO_MATCH, LINE_ITEM_MASTER, LINE_PRICE, LINE_UOM, LINE_QTY, LINE_REPEAT (WARN) | Date, Total, Lines |

**Thresholds** live in a versioned `RuleSetConfig` per legal entity; each invoice records the version it was checked against. Defaults: 90 days age, 1-day future grace, ±2% price, +2% over remaining PO balance, ±1 minor unit on totals (overridable per currency), description similarity ≥ 0.90, fuzzy duplicates within 30 days and edit distance ≤ 2, confidence ≥ 0.85.

**Money handling.** Amounts are integers in minor units using ISO 4217 exponents (LKR and USD 2, JPY 0, KWD 3). Multiplication uses `Decimal` with `ROUND_HALF_UP`; floats never touch an amount. Line prices are compared per the PO's unit of measure, converted through the item master.

**Routing.** `decide()` returns one of three routes:

- `AUTO_APPROVE` — every rule passes, every required field ≥ 0.85, and (if configured) the amount is under the auto-post ceiling.
- `REVIEW` — with the reasons: failed or skipped rule codes plus `LOW_CONF:<field>`.
- `BLOCKED` — a BLOCK rule failed; a person can only reject or confirm a duplicate.

**Approval gate.** Approve is allowed only with no BLOCK, no SKIPPED, and a valid reason code on every remaining FAIL (`OTHER` needs a 20-character comment).

**Queue priority** orders the review queue (breached SLAs pinned first):

priority = 40 × blocker + 25 × (SLA used ÷ SLA) + 20 × min(1, amount in LKR ÷ 10,000,000) + 15 × duplicate suspicion

Blocker is 0 or 1; duplicate suspicion is 1 for an exact match, 0.6 for a fuzzy one, otherwise 0. The SLA term keeps growing past 100% used, so an overdue task keeps rising (`libs/rules/decide.py`).

# 5. Exactly-once posting and failure handling

Double posting is stopped at four layers, and the only path into the ERP is a transactional outbox whose relay looks the invoice up in the ERP before every post. The worst case for any failure is a delay or a person, never a second payment or a silent loss.

## Duplicate layers

| Layer | Catches | Mechanism |
|---|---|---|
| Message | Kafka redelivers the same message | `inbox.message_id` primary key; acknowledged as a no-op |
| File | Same PDF sent twice | `inbox.pdf_sha256` unique; object store is content-addressed and write-once |
| Business document | Same invoice re-scanned as a new file | Partial unique index on `business_key` (supplier, invoice no., date, total, currency) where state ≠ REJECTED |
| ERP edge | Crash after the ERP accepted, before our DB update | Relay calls `find_by_external_ref(invoice_id)` before every post |

A colliding business key is stored, not rejected: the duplicate keeps its own invoice row with `business_key = NULL` and `duplicate_of` set, and goes to `DUPLICATE_SUSPECTED` for a person to compare. Dropping it would be the lost-invoice failure.

**Why an outbox.** Approving and "remember to post" must be one atomic fact. The approval transition and the `outbox` row commit in one transaction; the relay drains the outbox and is the only component allowed to call the ERP.

## Crash and error handling

| Failure | What happens | Result |
|---|---|---|
| Crash before the inbox write | Offset not committed; Kafka redelivers | No loss |
| Crash after the inbox write | Redelivery hits the unique key; acknowledged | No duplicate |
| Worker dies mid-extraction | 120 s lease expires; sweeper returns it to RECEIVED | Retried; extra tokens at worst |
| Model output fails the schema | One repair re-prompt, then the fallback model, then a manual-entry task | Never dropped |
| Quoted evidence not in the PDF text | Field confidence forced to 0 | Field outlined for review |
| Provider 429 / 5xx / outage | Shared token bucket (10% under caps), backoff with full jitter up to 6 attempts, circuit breaker at 50% failures | Backlog waits in the DB |
| Relay dies after the ERP accepted | Outbox row still pending or UNKNOWN; next pass finds the document by external ref | One ERP document |
| ERP business error | POST_FAILED, then NEEDS_REVIEW with the ERP message | Person decides |

| Failure | What happens | Result |
|---|---|---|
| ERP down | Back-off 1 min doubling to 30 min; invoice waits in APPROVED | No loss |

The relay crash case is proven by an automated chaos test: the fake ERP accepts, then times out; the retry finds the document and `post_invoice` is called exactly once.

**Daily reconciliation, the lost-invoice net.** Every guarantee above is checked once a day against an independent source (`libs/core/reconcile.py`). At 23:30 in each entity's time zone, the scheduler compares that business day three ways: the channel against the inbox, POSTED invoices against the ERP, and pipeline ERP documents against POSTED invoices. The comparison itself is a pure function (`compare`), unit-tested without I/O.

| Difference | Severity | Means |
|---|---|---|
| `MESSAGE_WITHOUT_INBOX` | Critical | The channel delivered a message that never reached the inbox: a lost invoice |
| `POSTED_WITHOUT_ERP_DOC` | Critical | We marked an invoice POSTED but the ERP has no document for it |
| `ERP_DOC_WITHOUT_POSTED` | Critical | The ERP holds a pipeline document whose invoice is not POSTED (for example an unhealed relay crash) |
| `DUPLICATE_ERP_DOC` | Critical | More than one ERP document for one invoice: a double payment |
| `DOC_NO_MISMATCH` | High | Our recorded ERP document number differs from the ERP's |
| `AMOUNT_MISMATCH` | High | The ERP document's total or currency differs from ours |

Each difference is stored as a `reconciliation_item`, written to the invoice's audit trail (`RECONCILIATION_DIFFERENCE`), paged (an ERROR log line in dev; PagerDuty or Opsgenie in production) and shown on the ops dashboard. A source that cannot be read never passes silently: a missing message ledger makes the run PARTIAL, and an unreachable ERP makes it ERROR, retried on the next 5-minute tick. Days the scheduler missed are caught up for 7 days back (Kafka keeps 14), and a finished day is never rerun. Supervisors can trigger a run from the dashboard; auditors can only read.

"What the channel delivered" must come from outside the inbox, or the check would compare the inbox with itself. Production reads the Kafka topic (`KafkaLedger`); dev reads a ledger file the demo seed writes (`MESSAGE_LEDGER=file|kafka|none`).

# 6. Review UI/UX

The UI is built around one number: reviewer time. At about 3 FTE per point of exception rate at month-end, every second saved per invoice matters, so the workspace puts evidence beside each value and makes the next action obvious.

## Four screens

| Screen | Who | What it does |
|---|---|---|
| Review queue | Reviewer | Tasks sorted breached-first, then by priority; SLA countdown pills and escalation chips (Warned, Supervisor pool, Finance mgr); reason chips; filters by entity, reason, SLA, amount band, supplier; "Next invoice" claims the top task for 15 minutes |

| Screen | Who | What it does |
|---|---|---|
| Review workspace | Reviewer | PDF left (55%) with evidence boxes, validation cards right (45%), pinned action bar |
| Checker view | Checker | Maker's decision, reason, comment, changed values (old struck through), overrides and rule results before → after; approve or send back only |
| Ops dashboard | Supervisor | Nine KPI tiles with drill-down; arrivals vs processed; queue depth by reason; reviewer throughput; backlog vs capacity; daily reconciliation panel (last 14 runs with their differences, run-now for supervisors) |

## Design principles, and how each is built

- **Evidence beside the claim.** Clicking a field outlines its source box on the PDF; clicking a box focuses the field. Low-confidence boxes are outlined amber.
- **Exceptions first.** A list at the top of the panel orders problems the way people fix them: identity, date, total, lines. J and K step through it.
- **Live re-validation.** Each edit calls `/validate` after a 300 ms pause; nothing is saved. Changed results animate once and are announced through an `aria-live` region ("Total reconciles now passes").
- **Never colour alone.** Every status pairs colour, icon and word: ✓ Pass, ▲ Warn, ✕ Fail, – Not checked.
- **Numbers built for checking.** Tabular numerals, right-aligned, with the currency code and signed differences (`Diff +0.50`).
- **Keyboard first.** J/K issues, E edit, A approve, O override, R reject, D duplicate, H hold, N next invoice, ? help.

## Workspace cards

- **Date card:** ISO date input with the long form below ("3 October 2026"), a "Confirm" button when day and month are ambiguous, and "Post in first open period" when the period is closed.
- **Total card:** a reconciliation strip, `Subtotal + Tax + Charges = computed · Invoice · Diff`, each term clickable to its evidence, with the remaining PO balance underneath.
- **Line items grid:** inline-editable cells; qty against open PO qty; unit price against PO price with variance %; a PO-line picker ranked by similarity; add, split and delete rows.
- **Action bar:** Approve (becomes "Send to checker" at LKR 1M), Approve with override, Reject, Mark duplicate, Hold – ask supplier, with the reason Approve is disabled always shown.

**States every screen handles:** loading skeletons; "Queue clear" with the next SLA risk; claimed by someone else (read-only, with their name); stale version (banner, draft kept); validate call failed (Approve disabled); ERP down (reference rules "not checked"); out-of-scope link (404); narrow screens (read-only).

**Visual system.** Neutral surfaces with one accent for primary actions; status colours that hold 4.5:1 contrast on their tints; base text 14 px (13 px compact); light, dark and system themes. Chart colours were checked with a colour-blindness validator in both themes.

**Screenshots** (demo data, Chrome, 1440 × 900)

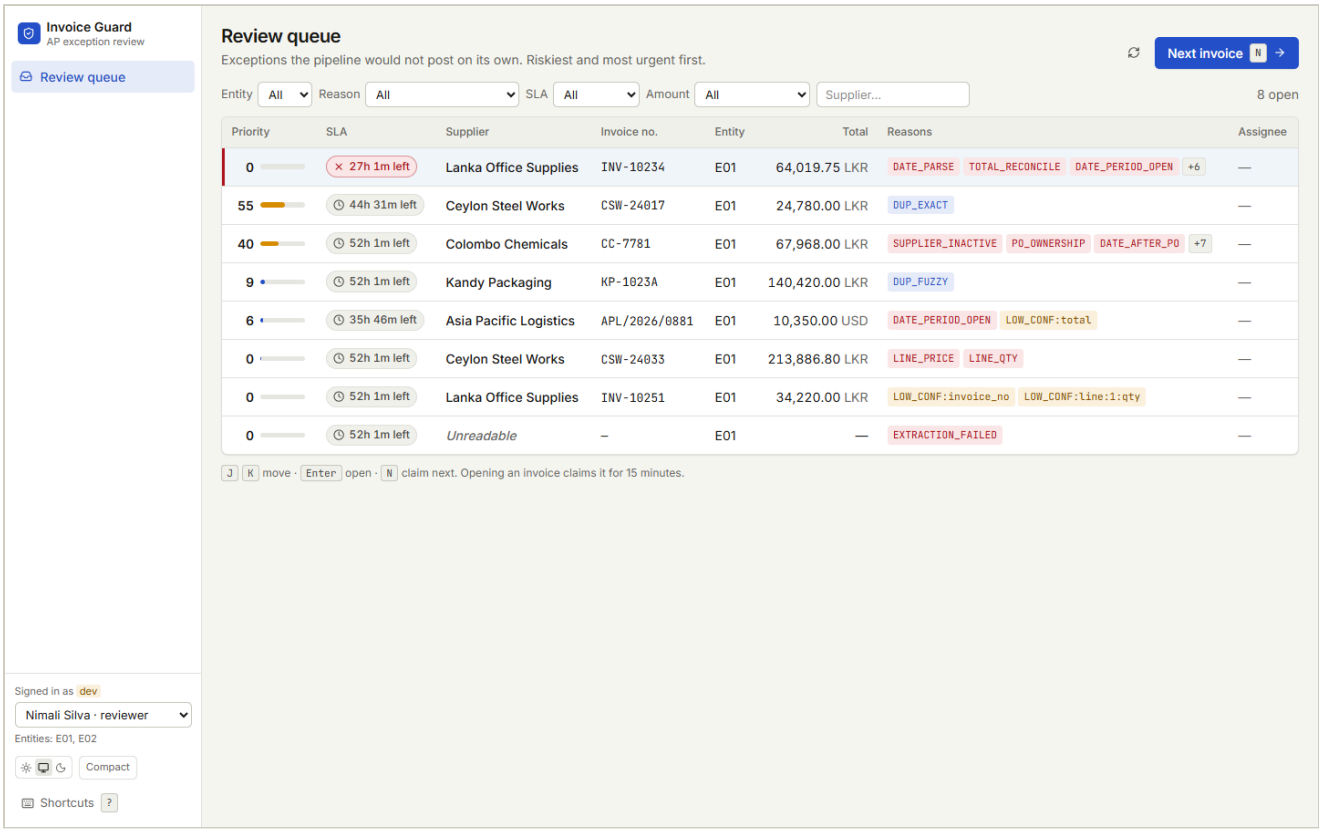



*Figure 3. Review queue: breached SLAs pinned first, then by priority; reason chips say why each invoice needs a person.*

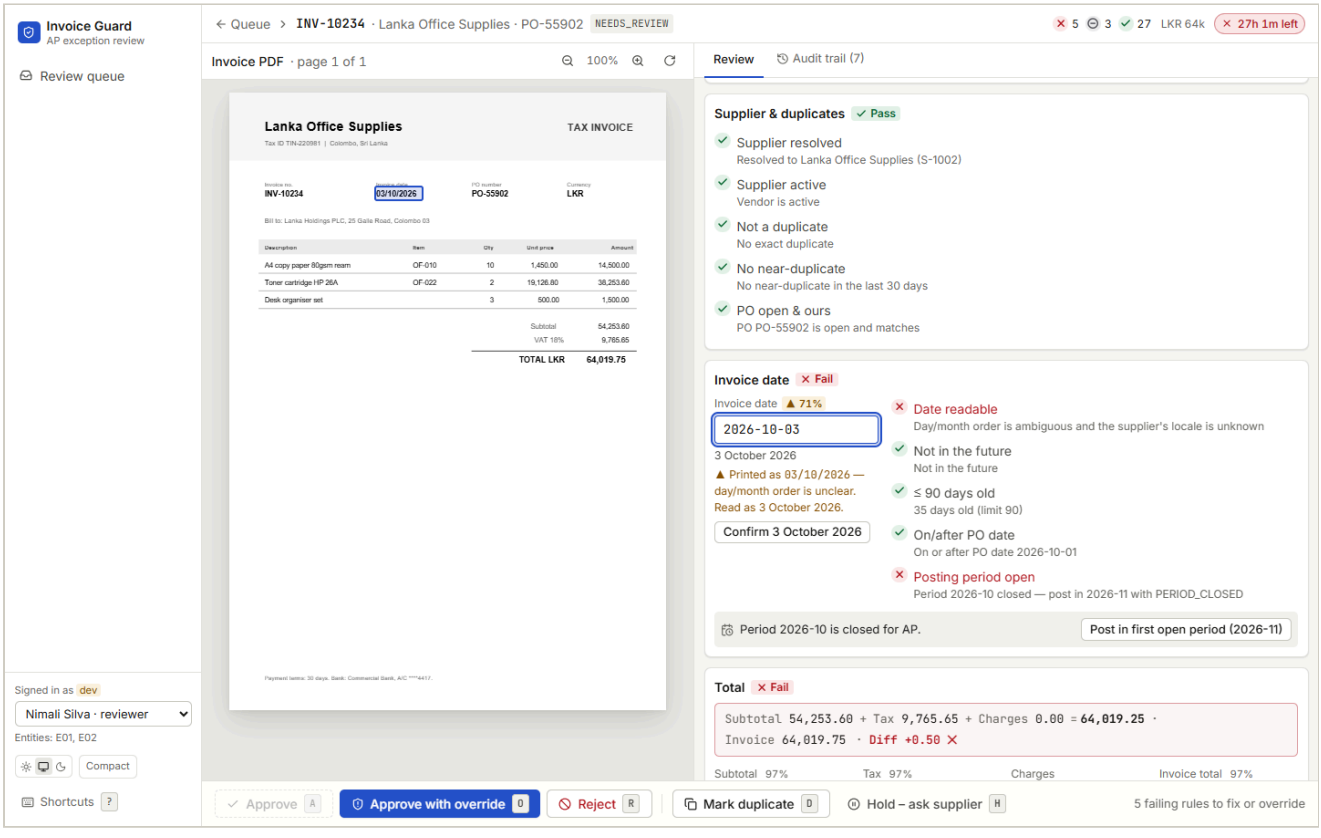



*Figure 4. Review workspace on the TSD's example invoice: the selected date's source box is outlined on the PDF, every failing rule is listed, and Approve stays disabled.*

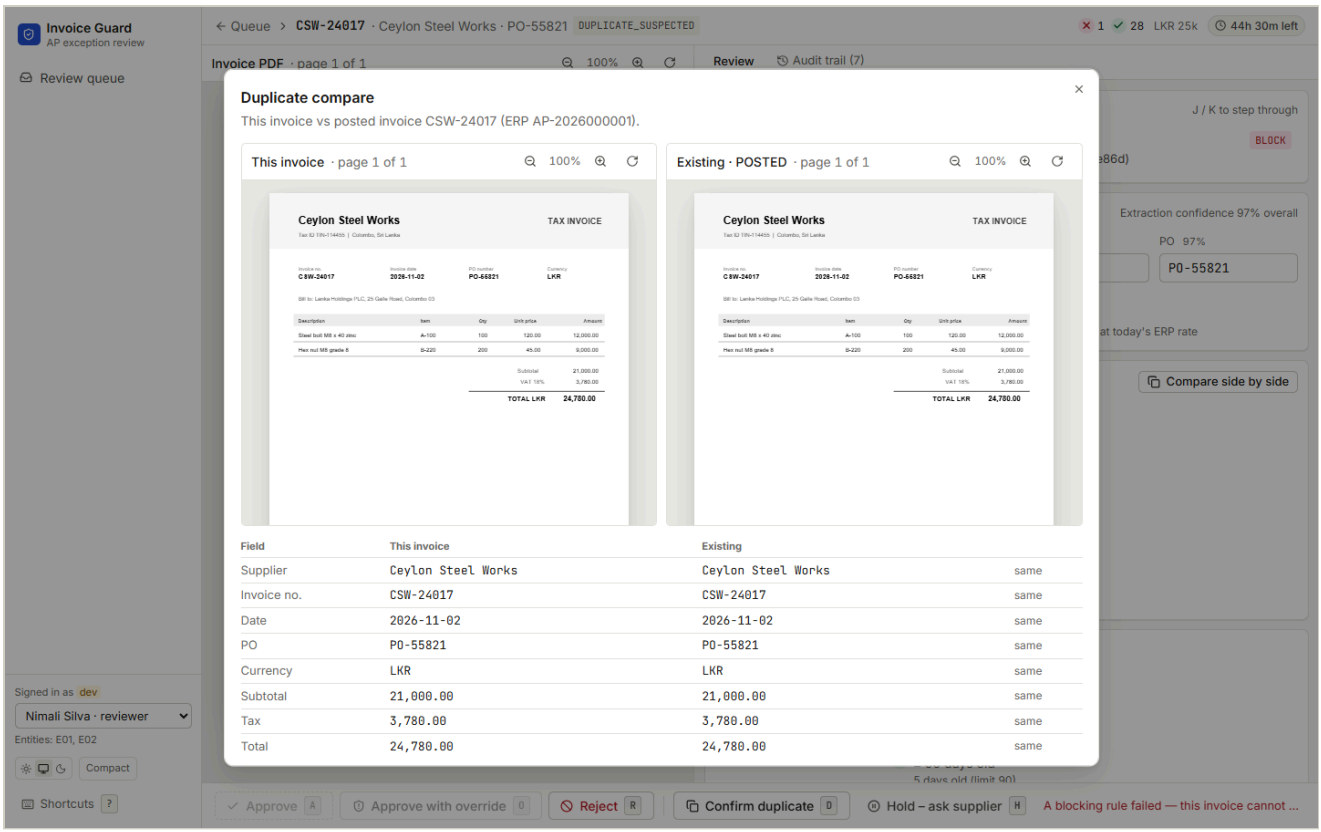



*Figure 5. Duplicate compare: a re-scan of a posted invoice, both PDFs side by side with a field-by-field diff.*

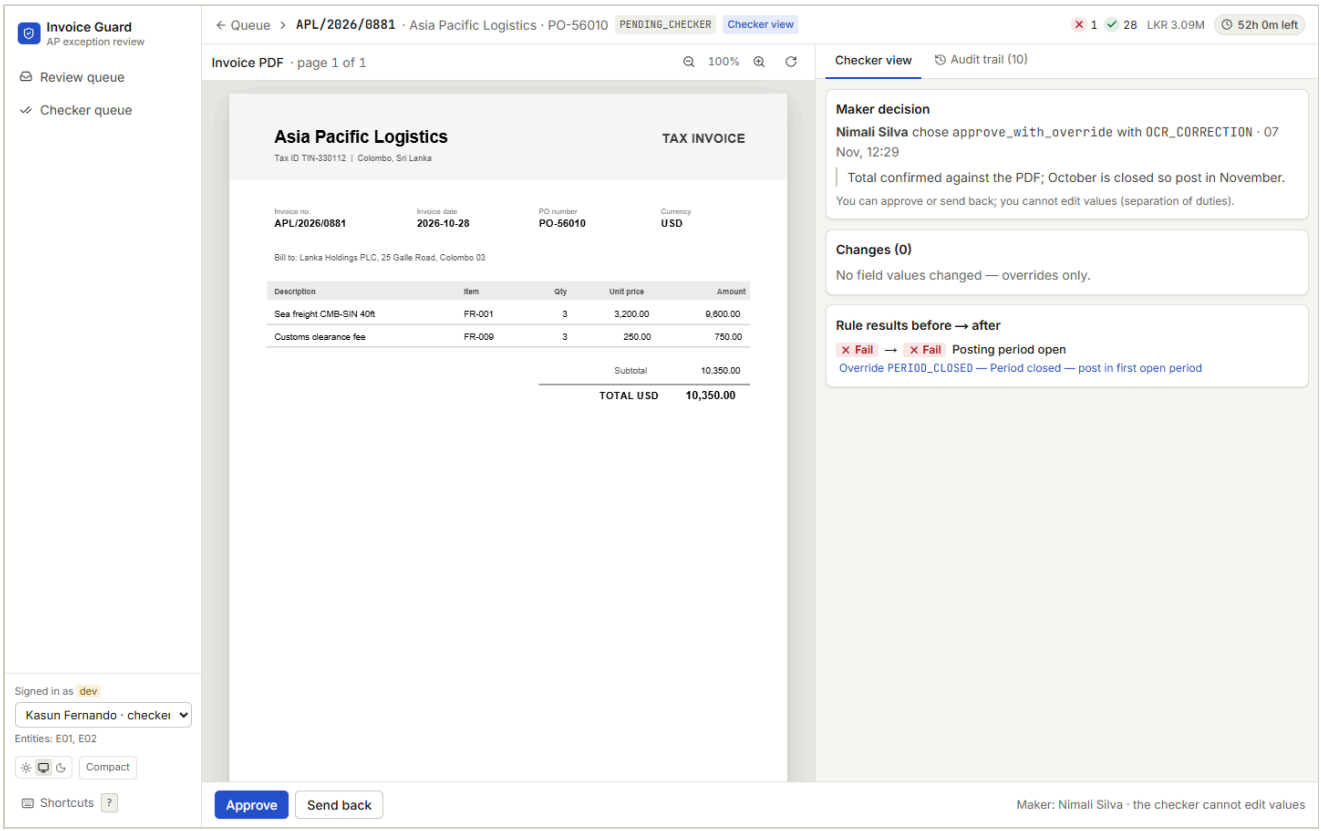



*Figure 6. Checker view: the maker's reason, comment and override; the checker can only approve or send back.*

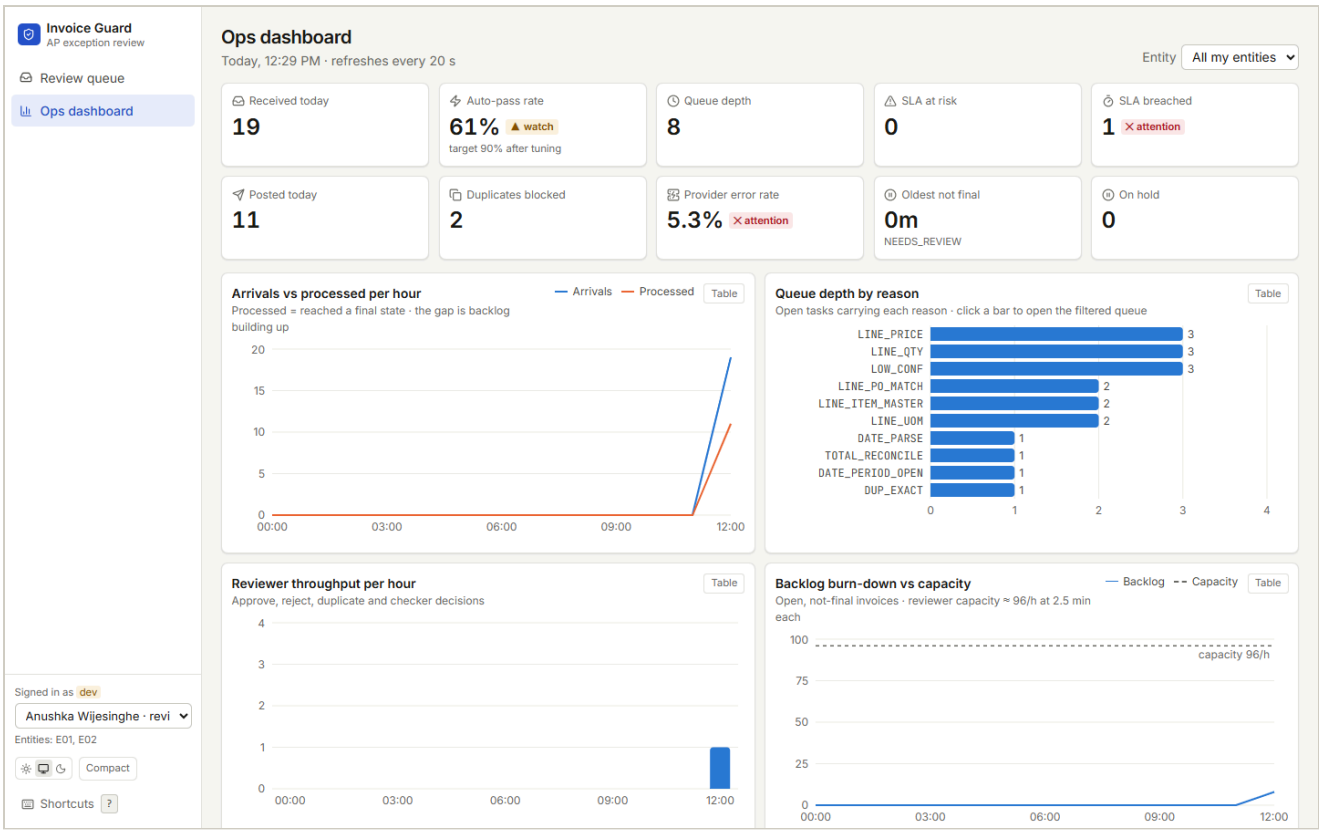



*Figure 7. Ops dashboard: KPI tiles with drill-down and four charts, each with a table view.*

# 7. Human controls and security

Every control is enforced on the server in `libs/core/review.py`, not in the UI: a person can change what posts, but never alone at or above LKR 1M, never without a reason code, and never outside their legal entities.

## Roles (scoped per legal entity)

| Role | Can | Cannot |
|---|---|---|
| Reviewer (maker) | Claim, correct fields, override FAIL rules with a reason, approve below LKR 1M, reject, mark duplicate, hold | Approve own work at or above LKR 1M; override BLOCK rules |
| Checker | Approve or send back maker decisions at or above LKR 1M | Edit values; check an invoice they made (`checker_id ≠ maker_id`) |
| Supervisor | Everything a reviewer can, plus the ops dashboard and invoices escalated to the supervisor pool | Bypass maker-checker |
| Auditor | Read everything in scope, including the event log | Any write |

## Decision rules

- **Maker-checker:** a manual approval of an invoice at or above LKR 1M equivalent (ERP daily rate) goes to `PENDING_CHECKER` and creates a checker task. Send back needs a comment and returns the task to the same maker.

- **Reason codes** on every override, correction and rejection: PERIOD_CLOSED, PRICE_AGREED, QTY_PARTIAL, SUPPLIER_CONFIRMED, OCR_CORRECTION, DUPLICATE_CONFIRMED, NOT_OUR_INVOICE, OTHER (comment of 20+ characters).
- **Server re-validates on approval** and refuses with 409 if the rule results changed since the reviewer's last check (for example, the PO was consumed).
- **Claims and versions:** opening an invoice claims it for 15 minutes; a second reviewer gets 409 "Claimed by …". Decisions carry `If-Match: <version>`; a stale version is refused with 409 and the draft is kept.
- **Entity scope** is applied in the repository layer; an invoice outside the user's entities returns 404, so its existence is not revealed.

**SLA and escalation.** Review SLA is 8 business hours (4 in the month-end window: last 3 and first 2 business days), counted on the entity's calendar (08:30–17:30 Asia/Colombo, Mon–Fri). The clock pauses while a task is on hold; a hold lasts at most 3 business days and then the task resumes on its own. The queue shows on track, at risk (≥ 75%), breached (≥ 100%) and on hold.

A scheduler (`services/scheduler`, logic in `libs/core/escalation.py`) runs every 5 minutes. Each level fires once and never goes down, so running it twice, or after an outage, never double-notifies:

| SLA used | Level | Action | Notified |
|---|---|---|---|
| 75% | 1 · Warned | Warning only | Assignee and team lead |
| 100% | 2 · Breached | Task moves to the supervisor pool: reviewers can't claim it (the current holder can finish); supervisors get it first from "Next invoice" | Supervisors and assignee |
| 150% (125% at month-end) | 3 · Second escalation | Stays with supervisors | Finance manager and supervisors |

Every pass also banks expired holds, pushes `sla_due_at` out by time spent on hold, and recomputes queue priority so the order reflects time burning. Notifications are rows in `notification`, unique per task and level, delivered by a pluggable sender (a log sender in dev) with up to 5 retries. The queue shows Warned, Supervisor pool and Finance mgr chips; an escalated invoice shows a red banner.

**Audit.** Every action writes an append-only `invoice_event` with actor, role, IP, from/to state and payload (changed fields old → new, reason code, comment, rule results before and after). Events are hash-chained per invoice; `verify_chain` detects tampering.

**Security.** The model has no tools and no posting rights; injection-like text in a PDF can at worst change values, which the rules then check. The controls below came out of two security assessments (section 9) and were merged to `main` in PR #8 (commit `2c6f49a`):

- **Identity fails closed.** `DEV_AUTH` defaults to off, so without an IdP the API returns 501. In dev, an `X-User` header picks a seeded user; a missing header is 401 and an unknown one 401. Interactive docs (`/docs`, `/openapi.json`) are served only with dev auth on.
- **PDF links.** HMAC-SHA256 signed with a dedicated `SIGNING_SECRET` (random per process when unset; never derived from another credential), valid at most `PDF_LINK_TTL_S` (300 s), compared in constant time. Object keys must be 64-character SHA-256 hex, and the resolved path must stay inside the store. PDFs are served with `default-src 'none'; script-src 'none'` and nosniff; pdf.js runs with `isEvalSupported: false`.
- **Response headers** on every API response: HSTS, nosniff, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, CSP `default-src 'none'; frame-ancestors 'none'`, `Cache-Control: no-store`.

- **Rate limit:** 600 requests per minute per client (`API_RATE_LIMIT_PER_MIN`), sliding window, in process. Unknown `X-User` values share one bucket, so rotating the header gains nothing. Exceeding it returns 429 with `Retry-After`.
- **Input bounds:** decision and checker comments at most 2,000 characters; corrections reject unknown fields; ORM queries only.
- **Posting guard:** the relay checks the required fields with an explicit error, not an `assert` (asserts vanish under `python -O`), and routes an incomplete invoice to review.
- **Supply chain:** CI runs `pip-audit`, Bandit, `npm audit --audit-level=high` and gitleaks; Dependabot covers pip, npm and Actions; `postcss` is pinned past its advisories.

## Key API endpoints

| Method + path | Purpose | Who |
|---|---|---|
| `GET /health` | Liveness (not rate limited) | Anyone |
| `GET /me` | Current user, roles, entities | Any user |
| `GET /users` | Dev identities for the switcher (404 when dev auth is off) | Dev only |
| `GET /reason-codes` | Valid reason codes | Any user |
| `GET /queue?kind=&entity=&reason=&sla=&amount=&supplier=` | Open tasks, breached first then priority, with facets | Reviewer, checker |
| `POST /queue/next` | Claim the top task (15-minute lease) | Reviewer, checker |
| `GET /invoices/{id}` | Header, lines, evidence, rule results, PO candidates, duplicate, task, events; `ETag` = version | In-scope users |
| `POST /invoices/{id}/claim` · `/release` | Take or give back the lease | Assignee or supervisor |
| `POST /invoices/{id}/validate` | Re-run the rules on draft corrections; nothing saved | Reviewer |
| `POST /invoices/{id}/decision` | approve, approve_with_override, reject, duplicate, not_duplicate, hold (`If-Match` required) | Reviewer (maker) |
| `POST /invoices/{id}/check` | Checker approve or send back (`If-Match` required) | Checker ≠ maker |
| `GET /invoices/{id}/pdf` | Signed URL, 5-minute expiry | In-scope users |
| `GET /files/{sha}.pdf?exp=&sig=` | The PDF itself, only with a valid signature | Link holder |
| `GET /ops/metrics` | KPI tiles and chart series | Supervisor, auditor |
| `GET /ops/reconciliation?entity=&limit=` | Latest runs, newest first, with differences | Supervisor, auditor |
| `POST /ops/reconciliation/run` | Run reconciliation now for one entity and day | Supervisor |

**Configuration** (environment or `.env`; business thresholds stay in the versioned `RuleSetConfig`):

| Setting | Default | Purpose |
|---|---|---|
| `DEV_AUTH` | `false` | Trust the `X-User` header; never in production |
| `SIGNING_SECRET` | empty (random per process) | PDF link key; from a vault in production |
| `PDF_LINK_TTL_S` | `300` | Longest a PDF link may live |
| `API_RATE_LIMIT_PER_MIN` | `600` | Requests per minute per client; `0` turns it off |
| `CORS_ORIGINS` | `http://localhost:3000` | Allowed browser origins |
| `ANTHROPIC_API_KEY` | empty (use demo sidecars) | Model access |
| `MODEL_PRIMARY` · `MODEL_FALLBACK` | `claude-haiku-5-5` · `claude-sonnet-5-5` | Extraction models |
| `LLM_RPM` · `LLM_TPM` | `500` · `400000` | Provider limits for the token bucket |
| `DATABASE_URL` | `sqlite:///./data/invoice_guard.db` | State DB (Postgres in production) |
| `OBJECT_STORE_DIR` | `./data/blobs` | PDF store |
| `KAFKA_BOOTSTRAP` · `KAFKA_TOPIC_*` | `localhost:9092` · `invoices.received/posted/dlq` | Messaging |
| `MESSAGE_LEDGER` | `file` | Reconciliation's channel source: `file`, `kafka` or `none` |

# 8. Implementation and testing

The system is one Python monorepo with four background processes (ingest, worker, relay, scheduler), a FastAPI review API and a Next.js app. It is verified by 117 automated tests, a demo seed that runs the real pipeline, a terminal walkthrough and a real-browser run of the UI.

## Stack

| Layer | Choice |
|---|---|
| UI | Next.js 15 (App Router), React 19, TypeScript, TanStack Query + Table, Tailwind 4, Radix dialogs, Recharts, PDF.js 5 (loaded unbundled) |
| API | FastAPI, Pydantic v2, SQLAlchemy 2, Alembic |
| Workers | Python asyncio; aiokafka for ingest |
| State DB | SQLite (WAL) in dev and pilot; Postgres 16 in production, same models |
| Extraction | Anthropic structured outputs (`messages.parse`); primary `claude-haiku-5-5`, fallback `claude-sonnet-5-5` |
| ERP | `ERPAdapter` protocol; fake ERP with fault injection for dev and tests |

### Tests (117, all passing in about 18 s)

| Suite | Tests | Covers |
|---|---|---|
| Rule engine | 53 | Every rule; TSD boundaries (exactly 90 days, ±1 minor unit, 2.0% vs 2.02%); UOM conversion; routing; approval gate; maker-checker; priority; Hypothesis property tests for money |
| Pipeline | 7 | Auto-post; Kafka redelivery and same-file no-ops; relay crash chaos test; ERP down; re-scan duplicate never posts; unreadable PDF becomes a task; audit chain detects tampering |
| Review API | 10 | Wireframe fix-override-approve posts in the open period; maker-checker separation; send back to the same maker; checker approve posts; entity-scope 404; stale version and foreign claim 409; reject needs a reason and BLOCK cannot approve; ops access; hold pauses the SLA and cannot stack; reconciliation API |
| SLA scheduler | 10 | Business-hour clock across nights and weekends; each level once and idempotent; missed levels raised after an outage; month-end 4 h SLA and 125% escalation; hold pauses then resumes with 27 h banked; priority rises with SLA used; supervisor pool and current-holder exception; delivery once; failing sender retries then gives up |
| Reconciliation | 16 | Clean day OK; each difference kind; doc number mismatch; lost message caught and paged; POSTED but missing in ERP; double posting caught; unhealed relay crash, then healed; missing ledger is PARTIAL, not OK; ERP down is ERROR and retried; runs after 23:30 local and catches up; finished days never rerun |
| Security | 21 | Dev auth off by default and fails closed; missing header 401; old default key rejected; expiry beyond TTL rejected; traversal and non-hash keys never reach the filesystem; checker comment bound; security headers; PDF keeps its own CSP; rate limit returns 429 and resists header rotation |

### Demo and walkthrough

- `scripts/seed_demo.py` generates 19 invoice PDFs with exact evidence boxes and runs them through ingest, worker, rules and relay: 11 auto-post, 7 go to review, 1 becomes a manual-entry task, plus one Kafka redelivery.
- `scripts/walkthrough.py` narrates one invoice end to end in the terminal: arrival, redelivery, extraction, rules, claim, correction, maker approval, checker approval, posting and the audit trail.
- CI runs three jobs. Backend: ruff, mypy, pytest and an Alembic migration on a fresh DB. Web: ESLint, type checks and a production build. Security: pip-audit, Bandit (medium and above), npm audit (production, high and above) and a gitleaks scan of the full history. Dependabot raises weekly updates for pip, npm and GitHub Actions.

# 9. Problems found while building, and their fixes

Eleven real defects surfaced from running the system rather than from reading the code: one in the rules, one in the model integration, five in the UI found by driving it in Chrome, three in the data layer, one in the demo data. All are fixed and committed.

| Found by | Problem | Fix |
|---|---|---|
| Demo seed | `LINE_PRICE` compared a per-bundle price with the PO's per-each price and reported +2400% | Convert to the PO's unit of measure through the item master before comparing |
| Claude API check | Forced `tool_choice` returns 400 on current models | Structured outputs via `messages.parse(output_format=…)` |

| Found by | Problem | Fix |
|---|---|---|
| Browser run | Workspace crashed: Next's bundler re-bundling `pdfjs-dist` v5 throws `Object.defineProperty called on non-object` | Dropped `react-pdf`; PDF.js loads unbundled as a native ES module from `/public/pdfjs` |
| Browser run | Hydration mismatch on every page: identity lives in `localStorage`, which the server can't see | The app renders after mount |
| Browser run | A reviewer could not confirm an ambiguous date that was already right: no change meant no correction was sent | "Confirm 3 October 2026" button; ISO text input in place of the locale date picker, which showed `10/03/2026` |
| Screenshots | The page scrolled when a field got focus, pushing the header off screen: screen-reader-only text escaped the scrolling panel and stretched the document to 2224 px | Scroll containers are positioned and clipped; the page never scrolls |
| Screenshots | A card still said "Fail" after its only failure was overridden; a confirmed date still showed 71% confidence | "Pass · with override"; confirmed or corrected fields show 100% |
| SQLite | SQLite drops `tzinfo`, breaking lease and SLA comparisons | `UTCDateTime` column type: always aware UTC in Python |
| SQLAlchemy | Replacing invoice lines hit `UNIQUE(invoice_id, line_no)` because inserts flush before deletes | Flush the deletes before re-inserting |
| Migration review | The generated SLA migration added a NOT NULL column with no default (fails on any database with review tasks), and tasks already on hold would silently come off hold | Server default 0, plus a backfill that starts the pause for held tasks; upgrade tested on a populated database |
| Demo seed | The seed aged tasks by wall-clock hours, so on a weekend the business-hours clock showed almost no escalations | Seed ages tasks in business time to 80%, 110% and 160% of SLA, then runs one scheduler pass |

**Security testing.** Two assessments followed the build. A grey-box security test (Nov 7) found nine issues, all now fixed and covered by regression tests. An OWASP Top 10 assessment (Nov 8) found eleven more, all open and awaiting approval. Full detail: Security Test Report and OWASP Security Assessment.

| ID | Severity | Problem | Fix (commit `2c6f49a`) |
|---|---|---|---|
| F1 | High | PDF links signed with a public default key; `sha` not validated, so a Windows backslash path read files outside the store | Dedicated key, hash-only keys, path containment, TTL cap |
| F2 | High | `DEV_AUTH` on by default; a missing header became `n.silva` | Off by default, fails closed; missing header 401 |
| F3 | High (advisory) | `postcss` advisories through Next.js | npm override to `^8.5.32` |
| F4 | Low | No security headers on API responses | Header middleware |
| F5 | Low | Checker comment unbounded | 2,000-character limit |
| F6 | Low | No rate limiting | Per-client limit, 600 per minute |
| F7 | Info | API docs public | Only with dev auth |
| F8 | Info | CI ran lint only | Security CI job |

| ID | Severity | Problem | Fix (commit `2c6f49a`) |
|---|---|---|---|
| F9 | Medium | Dependabot pointed at folders that do not exist, so it never scanned pip or npm | Paths fixed |

Open from the OWASP assessment (no code changed yet): N1 high, the ingest consumer opens any file path named in a Kafka message; N2–N4 medium, dev compose services exposed on every interface, no clickjacking protection on the UI, and no input size limits; N5–N8 low (security-event logging, CI action pinning, Dependabot cooldown, dev-only npm advisories); N9–N11 informational (unkeyed audit chain, dev identity in the browser, broad CORS). The OWASP ZAP dynamic scan is still to run.

# 10. Decisions, known gaps and next steps

Phase 1 proves the guarantees on SQLite with a fake ERP, and daily reconciliation now checks them every night. The biggest remaining risks are unproven real-world integrations (SSO, Postgres, the model, a real ERP, the Kafka ledger) and the open OWASP finding on the ingest path.

## Decisions taken where the TSD is open

- **DUP_EXACT is BLOCK.** A reviewer can confirm the duplicate or reject; "Not a duplicate" applies to near duplicates, or after the reviewer corrects the business key.
- **SKIPPED blocks approval** and cannot be overridden: you can't override a check that never ran.
- **Auto-post ceiling** (`auto_approve_ceiling_lkr_minor`) is a setting, off by default. It serves the pilot's LKR 250k limit and the TSD's open question on always reviewing invoices above LKR 10M.
- **Non-PO invoices and credit notes** stay exceptions (PO_OWNERSHIP and TOTAL_POSITIVE fail) until their own flows exist.
- **A reconciliation that cannot read a source is not a pass.** It records PARTIAL or ERROR and says which check was skipped, because a silent OK would hide exactly the loss it exists to catch.
- **Dev conveniences fail closed.** Dev auth, the user switcher and the API docs are off unless switched on explicitly, so a deploy that forgets a setting is locked rather than open.

## Known gaps

| Gap | Impact | Next step |
|---|---|---|
| Ingest trusts the message's file path (OWASP N1) | A Kafka producer can pull any server file into review | Spool-directory allow-list, `%PDF` and size checks, validate `entity_id`, authenticate producers |
| No SSO | Dev auth now fails closed, but production has no login yet | OIDC (Entra ID or Okta), roles and entities from group claims |
| No email or Teams delivery | SLA warnings and escalations are recorded and logged, not delivered | An SMTP or Microsoft Graph sender; resolve role@entity recipients from the directory |
| No pager integration | Reconciliation alerts are ERROR log lines | PagerDuty or Opsgenie `Pager` |
| Kafka ledger untested | Reconciliation's channel check has run only on the dev ledger file | Verify `KafkaLedger` against the real topic in the shadow run |

| Gap | Impact | Next step |
|---|---|---|
| Postgres untested | `SKIP LOCKED` and partial unique indexes proven only on SQLite | Postgres in docker-compose and a CI job |
| Real extraction untested | Demo uses pre-recorded extractions; eval scoring not written | Run the 500-invoice eval set through the Anthropic extractor; gate on 98% field accuracy at ≥ 0.85 |
| Fake ERP never consumes PO balances | Repeat invoices against one PO keep passing LINE_QTY | Track invoiced quantity, or read it from the real ERP |
| Limits are per process | Token bucket and API rate limit each count one process only | Redis-backed bucket; rate limit at the gateway |
| Thresholds live in code | A change needs a deploy, not the TSD's two-admin approval | Versioned rule sets per entity in the DB |
| No holiday calendar | SLA business days skip weekends only, so a public holiday burns SLA | Per-entity holiday list in the SLA calendar |
| Open OWASP items N2–N11 | Dev infra exposure, UI clickjacking, no input size limits, logging and CI hardening | Fixes proposed in the OWASP assessment, awaiting approval |
| Not built | OCR for scanned pages, OpenTelemetry, S3/Blob store, Playwright and axe-core in CI, OWASP ZAP in CI | Phases 2–3 |

## Next steps, in order

1. Approve and fix OWASP N1, then N2–N4; run the OWASP ZAP scan.
2. Real delivery for SLA notifications and a pager for reconciliation differences.
3. Postgres in CI, then OIDC SSO: prerequisites for the single-entity pilot.
4. Extraction evals on real invoices; tune the confidence threshold before the shadow run, and verify the Kafka ledger there.
5. First real ERP adapter, confirming whether it supports native idempotency or only external-reference lookup.